

Capítulo 1

Implementación del Software

1.1 Arquitectura general

El diseño estructural del software se fundamenta en una separación de responsabilidades, que permite desarrollar y evaluar de forma independiente cada parte del sistema. Con este objetivo, se adoptó el patrón de diseño arquitectónico MVP, el cual divide el sistema en tres capas lógicas bien definidas: la Vista, el Modelo y el Presentador. Esta organización asegura el desacoplamiento de componentes, facilitando la escalabilidad del sistema ante la incorporación de otros modelos de osciloscopios o métodos adicionales de análisis.

La división de tareas facilita la validación algorítmica y control de la incertidumbre matemática especificados en la norma [1], permitiendo someter los algoritmos a ensayos automatizados, aislados de cualquier interfaz gráfica de usuario.

Figura 1.1: Diagrama de bloque de la arquitectura.

1.1.1 Capa de presentación: Vista

La **Vista** maneja de forma exclusiva la representación visual del sistema y la captura de las interacciones del operador. Ya que, solo declara estáticamente la disposición y propiedades de los componentes gráficos.

Esta capa se caracteriza por no tener referencias directas hacia la lógica de adquisición física o los algoritmos de procesamiento digital de señales. Ante cualquier interacción física del usuario, como el accionamiento de botones o la inserción de datos,

la Vista emite eventos en forma de señales de Qt (**Qt Signals**), que son interceptadas por el Presentador para desencadenar el flujo correspondiente.

1.1.2 Capa de lógica y datos: Modelo

El **Modelo** representa el núcleo funcional del software, abarcando el control del instrumental de laboratorio, la lógica de cálculo metrológico y la administración de la persistencia de datos.

Esta capa se subdivide en tres submódulos independientes especializados:

1. **Abstracción de hardware:** Implementada en el módulo **GWInstekGDS1000AU**. Encapsula todas las operaciones de control, configuración de canales y adquisición de datos de la memoria del osciloscopio, mediante comandos SCPI, usando el protocolo de comunicación PyVISA.
2. **Persistencia y gestión de archivos:** Administrada por el módulo **FileManager**. Maneja la creación y el acceso al sistema de archivos. Su función principal es el almacenamiento en bases de datos utilizando el formato HDF5. Asimismo, gestiona la exportación de resultados a hojas de cálculo tipo CSV o XLSX.
3. **Procesamiento digital de señales:** Llevado a cabo por el módulo matemático **LightningImpulseAnalyzer**. Constituye el motor matemático del sistema. Es responsable de acondicionar la señal y extraer sus parámetros característicos según la norma [2].

1.1.3 Capa de coordinación: Presentador

El **Presentador**, es implementado en el módulo **MainPresenter**, y actúa como coordinador de la arquitectura MVP. Tiene como responsabilidad orquestar la interacción entre la interfaz visual y los motores lógico-físicos.

1.2 Controlador de *hardware*

En esta sección se expone la implementación controlador del osciloscopio, cuya función general es abstraer las operaciones de bajo nivel, permitiendo desacoplar la comunicación con el dispositivo del resto del sistema.

1.2.1 Interfaz de comunicación y capa de abstracción de hardware

A nivel de *software*, para evitar el acoplamiento directo del código de la aplicación con las llamadas al sistema de bajo nivel (capa física de transmisión), se implementa una Capa de Abstracción de Hardware (HAL, *Hardware Abstraction Layer*), bajo la Arquitectura VISA. Esta capa de abstracción permite que los comandos de consulta (*queries*) y configuración (*writes*) permanezcan independientes de la interfaz de hardware utilizada (sea USB, GPIB, Ethernet o RS-232), permitiendo actualizaciones de la instrumentación sin necesidad de modificar el código del presentador.

El controlador inicializa el administrador de recursos, y ejecuta un escaneo automático para buscar algún dispositivo conectado, que tenga un descriptor de recurso lógico VISA, como el siguiente:

ASRL1 : INSTR

Si encuentra al menos un dispositivo direccionable, se conecta al mismo. La comunicación inicia instanciando el administrador de recursos VISA de Python (PyVISA), acompañado de la biblioteca `pyvisa-py` para asegurar un entorno multiplataforma sin depender de bibliotecas nativas propietarias.

Sobre el canal de transporte establecido con VISA, el software envía comandos SCPI, que debe cumplir con el siguiente formato indicado en el manual del fabricante [3]:

- **No sensibilidad a mayúsculas:** Puede usarse mayúsculas y minúsculas.
- **Separación de parámetros:** Al configurar parámetros, debe separarse el comando y su argumento numérico, mediante un carácter de espacio en blanco simple.
- **Finalizador de mensaje:** Cada instrucción enviada al osciloscopio debe finalizar con el carácter delimitador de fin de mensaje *Line Feed* (**LF**, en hexadecimal **0x0A** o a la secuencia de escape **\n**), para que el instrumento proceda a interpretar y ejecutar la orden.

Para gestionar la configuración del osciloscopio de manera robusta, se implementa una arquitectura, basada en métodos de consulta (*getters*) y configuración (*setters*), que encapsulan los comandos SCPI.

Los métodos de consulta usan la instrucción **.query()**, reciben una cadena de caracteres, que deben convertir al tipo de dato correspondiente (`float`, `int` o `string`) y la retornan al presentador. En tanto que, los métodos de configuración emplean la instrucción **.write()**, sin esperar ninguna respuesta del instrumento.

Este flujo se protege con una rutina de control de excepciones que, ante fallos críticos en el bus o pérdidas de sincronismo, aborta la operación y libera los recursos del sistema de forma segura, ya que de lo contrario impediría la reconexión posterior del instrumento, debiendo reiniciar la computadora o el osciloscopio.

A continuación, se ejemplifican los métodos de consulta y configuración de la escala vertical de un canal.

```
# Método de consulta.
def get_channel_scale(channel):
    scale = dso.query(f':channel{channel}:scale?')
    v_scale = float(scale)
    return v_scale

# Método de configuración.
def set_channel_scale(channel, value):
    dso.write(f':channel{channel}:scale {value}')
```

Por otro lado, a nivel físico, la conexión se realiza a través de un puerto USB 2.0 tipo B (esclavo) ubicado en el panel posterior del instrumento, y un puerto USB tipo A (maestro) de la computadora. Aunque, si bien el equipo se conecta a través de un puerto USB, la computadora lo detecta como un dispositivo de comunicación serie virtual¹, bajo la Clase de Dispositivo de Comunicación con Modelo de Control Abstracto (CDC-ACM, *Communication Device Class - Abstract Control Model*), que aparece en el administrador de dispositivos con el siguiente nombre [3]:

COM1

Donde COM1 identifica el número de puerto serie virtual asignado por el sistema operativo.

1.2.2 Sistema de Adquisición y Almacenamiento

La adquisición del impulso de alta tensión desde el generador Marx, hasta el almacenamiento en la computadora, se estructuran de la siguiente forma:

1. Acondicionamiento analógico.
2. Captura de impulso
3. Digitalización.
4. Desempaquetado de datos.
5. Reconstrucción y escalamiento de la onda.

¹El controlador (*driver*) USB, está disponible en la página del fabricante [4].

Acondicionamiento analógico

La señal eléctrica, debe ser adaptada en amplitud antes de ingresar al osciloscopio, para no dañarlo.

Para el canal de tensión (CH1), la señal se deriva utilizando un divisor de tensión resistivo, acoplado en cascada serie con un atenuador coaxial, por lo que el factor de atenuación equivalente resulta:

$$A_{CH1} = \text{Divisor} \times \text{Atenuador} \quad (1.1)$$

Para el canal de corriente (CH2), la descarga atraviesa una resistencia shunt conectada en serie con el elemento bajo ensayo, de la cual se toma su tensión a bornes, acoplada a un atenuador que reduce su magnitud a niveles adecuados.

$$A_{CH2} = R_{\text{shunt}} \times \text{Atenuador} \quad (1.2)$$

Ambas señales llegan al instrumento por medio de cables coaxiales de $50\ \Omega$, para reducir acoplamientos electromagnéticos parásitos (EMI).

Captura de impulso

La captura de impulsos ($1.2/50\ \mu\text{s}$) requiere que el osciloscopio opere en modo de disparo único (*Single Trigger*), dado que es un transitorio no periódico.

Cuando el operador inicia el modo de captura desde la GUI, el presentador crea y ejecuta un subproceso asíncrono, que mediante el controlador de *hardware*, realiza un sondeo periódico (*polling*) al osciloscopio, para saber el estado de disparo del mismo.

Al detectar el impulso finaliza el bucle de *polling* y emite una señal, que es capturada en el hilo de ejecución principal por un receptor (*slot*) del presentador, y finaliza el subproceso.

Como el instante de descarga es indeterminado, se implementa un hilo de ejecución secundario asíncrono, dedicado a consultar el estado de disparo, ya que si se ejecutara en el hilo principal, sería una operación bloqueante, que congele la aplicación e inhabilite la GUI. Por lo que, así se mantiene siempre reactivo el hilo principal de eventos gráficos, permitiendo al usuario manipular oscilogramas previos, cargar campos o cualquier otra acción, mientras el sistema espera la captura del impulso.

Digitalización

Las señales eléctricas que ingresan a los bornes analógicos del osciloscopio, se muestrean a una tasa que depende de la base de tiempo utilizada. Después, el convertidor analógico-digital (ADC) realiza la cuantización en amplitud, dividiendo el rango dinámico vertical en $2^8 = 256$ niveles de tensión discretos (8 bits). A continuación, se graban en la memoria del instrumento.

Desempaquetado de datos

Luego de que el osciloscopio digitalice la señal, el presentador ordena la descarga de los datos brutos (*RAW*) de los canales habilitados, y recibe un bloque de datos, compuestos por el encabezado y el vector de datos de la onda, que cumple con la siguiente estructura² [3]:

A B C D E

#: [1 byte] Caracter ASCII que indica el inicio de la cabecera.

A: [1 byte] Dígito ASCII que especifica la cantidad de caracteres que componen el campo B. Para adquisiciones estándar toma el valor ASCII '4', mientras que para adquisiciones de memoria extendida toma el valor '7'.

B: [4 o 7 bytes] Cadena de caracteres ASCII que representa el tamaño total en bytes del bloque de datos que le sigue (C+D+E). Para adquisiciones estándar toma el valor ASCII '8008', mientras que para adquisiciones de memoria extendida puede tomar los valores '2000008' o '4000008'.

C: [4 bytes] Intervalo de tiempo (Δt) entre dos muestras consecutivas de la señal, en formato de punto flotante de precisión simple según IEEE-754 (32 bits), en orden *little-endian* (se envía primero el byte menos significativo):

$$\Delta t = t[n + 1] - t[n]$$

D: [4 bytes] Bloque de datos no utilizado.

E: [8×10^3 , 2×10^6 o 4×10^6 bytes] Vector de datos de la onda, cuya longitud es definida en el campo B. Cada valor entero (base 10) es expresado con 2 bytes (16 bits), y transmitidos en orden *big-endian* (se envía primero el byte más significativo).

²La estructura de datos descrita en el manual del fabricante tiene algunas diferencias con los resultados obtenidos del instrumento, debido a la dificultad que esto presentó al momento de intentar procesarla, se contactó con el fabricante obteniendo un script de procesamiento del paquete de datos, que se corresponde con el formato indicado en este trabajo. A su vez, se encontró que el manual de programación de la serie GDS-1000B aporta detalles explicativos sobre cómo procesar esta estructura [5]

Una vez finalizada la descarga de datos, el presentador ordena al controlador de *hardware* que decodifique cada bloque acorde a su formato, considerando la estructura recién mencionada.

Reconstrucción y escalamiento de la onda

Con el vector de valores enteros brutos decodificado, primero se aplica la ecuación (1.3) para reconstruir la forma de onda que llegó a bornes del osciloscopio, considerando la escala vertical configurada al momento de la captura, y el Factor AD = 25 definido en el manual de programación del fabricante [3]:

$$\text{Onda sin escalar}[n] = \text{Datos brutos}[n] \times \frac{\text{V/div}}{\text{Factor AD}} \quad (1.3)$$

A continuación, para obtener las magnitudes reales aplicadas al objeto bajo ensayo, se multiplica escalarmente los vectores de tensión y corriente, por sus respectivos factores de atenuación, cargados por el usuario en los campos pertinentes de la ventana, aplicando la siguiente ecuación:

$$\text{Onda escalada}[n] = \text{Onda sin escalar}[n] \times A_{\text{CH<X>}} \quad (1.4)$$

Los vectores resultantes de tensión (en volts, V) y de corriente (en amperes, A), se transfieren al módulo de procesamiento matemático para determinar sus parámetros normativos: tiempo de frente T_1 , tiempo de cola T_2 , valor de cresta U_t y sobreimpulso OS, conforme a los requerimientos de la norma [2].

1.3 Motor de Cálculo: Procesamiento de Señales y Ajuste de Curvas

Este componente constituye el núcleo matemático del software, y es responsable de obtener los parámetros normativos a partir de los arreglos temporales de tensiones discretas adquiridas por el *hardware*.

El motor de cálculo ha sido diseñado exclusivamente para cumplir con las normas IEC 60060-1 [2] e IEC 61083-2-1 [1], aplicando criterios de modularidad, que permiten su validación automatizada e independiente del resto de la aplicación.

1.3.1 Flujo algorítmico de cálculo

Este proceso se divide en tres fases operativas: preprocesamiento común, clasificación dinámica y reconstrucción de la señal.

Fase I: acondicionamiento inicial

Esta etapa es común a todos los registros adquiridos.

1. **Eliminación de offset:** Se calcula el valor de tensión continua (V_{offset}) promediando estadísticamente una ventana de datos adquiridos ($N_{\text{pretrigger}}$) en el instante previo al disparo (“pre-trigger”), donde la curva registrada ($U_e[n]$) es nula.

$$V_{\text{offset}} = \frac{1}{N_{\text{pretrigger}}} \sum_{n=1}^{N_{\text{pretrigger}}} U_e[n] \quad (1.5)$$

El vector de tensión corregida se calcula como:

$$U_{\text{corregida}}[n] = U_e[n] - V_{\text{offset}} \quad (1.6)$$

2. **Normalización de polaridad:** Dado que los impulsos pueden ser de polaridad positiva o negativa, el algoritmo detecta el sentido del impulso evaluando el signo de la muestra extrema:

$$\text{polaridad} = \text{sign}(\arg \max |U_{\text{corregida}}[n]|) \quad (1.7)$$

El vector de tensión corregida se multiplica por este término para simplificar el resto del procesamiento, asegurando que el análisis sea exclusivamente en el semiplano positivo:

$$U_{\text{abs}}[n] = \text{polaridad} \cdot U_{\text{corregida}}[n] \quad (1.8)$$

3. **Normalización unitaria:** Se normaliza la amplitud al dominio unitario respecto a su valor de cresta.

$$U_{\text{norm}}[n] = \frac{U_{\text{abs}}[n]}{\max(U_{\text{abs}}[n])} \quad (1.9)$$

Fase II: Bifurcación de análisis e identificación de parámetros base

En este punto, el flujo se divide según si debe procesar un impulso de referencia (pleno) o un impulso de ensayo, que puede ser pleno o cortado.

Caso A: Análisis de impulso de referencia (pleno)

Se asume que la onda es plena y se procede al ajuste paramétrico.

4. **Segmentación de datos:** Se selecciona una ventana de datos en el intervalo comprendido entre el 20 % en el flanco de subida y el 40 % en la cola de descarga del impulso.
5. **Ajuste de curva base:** Se realiza un ajuste paramétrico no lineal sobre la ventana de muestras, utilizando el algoritmo de **Levenberg-Marquardt**, teniendo

como modelo de estimación la función de doble exponencial:

$$u_d[n] = U \cdot \left(e^{-\frac{t-t_d}{\tau_1}} - e^{-\frac{t-t_d}{\tau_2}} \right) \quad (1.10)$$

Donde los coeficientes desconocidos a determinar son:

- U el parámetro de amplitud escalar.
- τ_1 la constante de tiempo de descarga.
- τ_2 la constante de tiempo de carga.
- t_d el retardo temporal del origen virtual.

Caso B: Análisis de ensayo (LI vs. LIC)

Si evalúa una onda de ensayo, se inyectan los parámetros de la onda de referencia procesada en el **Caso A**, y se ejecuta el siguiente análisis:

6. **Cálculo del desfase temporal:** El corrimiento temporal, t_L , entre el frente de onda del último impulso con el de referencia, se calcula como el promedio de las diferencias de los instantes de tiempo, correspondientes al 30 %, 50 % y 80 % de U_t respectivamente, de ambos impulsos.

$$t_L = \frac{(t_{30,\text{ref}} - t_{30,i}) + (t_{50,\text{ref}} - t_{50,i}) + (t_{80,\text{ref}} - t_{80,i})}{3} \quad (1.11)$$

7. **Alineación del eje temporal:** Se obtiene el vector temporal alineado del impulso de ensayo, $t_{\text{aligned}}[n]$, partiendo de su vector temporal original $t[n]$, al que se lo desplaza la cantidad t_L .

$$t_{\text{aligned}}[n] = t[n] + t_L \quad (1.12)$$

8. **Clasificación del impulso:** Se calcula la diferencia absoluta punto a punto entre el impulso de ensayo y el de referencia, usando sus vectores normalizados en amplitud.

$$|U_{\text{norm},i}[n] - U_{\text{norm},\text{ref}}[n]| > \text{umbral} \quad (1.13)$$

En caso de que la desviación no supere el umbral, la onda es clasificada como **impulso pleno (LI)** y el flujo se dirige al **Caso A**.

Si la desviación supera el umbral, la onda se clasifica como **impulso cortado (LIC)** y se procede a analizar el colapso de la onda.

9. **Cálculo del factor de escala:** Se calcula como el promedio del intervalo comprendido entre el 30 % y 80 % de $U_{\text{corregida}}[n]$ del frente de la onda de ensayo y la de referencia.

$$\bar{U}_i = \frac{1}{N} \sum_{n=k_{30\%}}^{k_{80\%}} U_i[n] \quad (1.14)$$

$$\overline{U_{\text{ref}}} = \frac{1}{N} \sum_{n=k_{30\%}}^{k_{80\%}} U_{\text{ref}}[n] \quad (1.15)$$

$$E = \frac{\overline{U_i}}{\overline{U_{\text{ref}}}} \quad (1.16)$$

10. **Escalado de la curva base:** Debido a que la descarga disruptiva hace colapsar el ajuste por el corte abrupto de la señal, se sintetiza la curva base de la onda de referencia $U_{b,\text{ref}}[n]$, y se escala en amplitud con el factor de escala E .

$$U_{b,i}[n] = E \cdot U_{b,\text{ref}}[n] \quad (1.17)$$

11. **Tiempo de corte:** Se analiza la cola del impulso, minimizando la derivada primera para localizar el sector de máxima pendiente de caída:

$$n_{\text{abrupto}} = \arg \min \left(\frac{dU_{\text{cola}}[n]}{dt} \right) \quad (1.18)$$

El inicio del colapso se encuentra evaluando el punto de inflexión local mediante la derivada segunda:

$$n_{\text{inflexion}} = \arg \min \left(\frac{d^2 U_{\text{cola}}[n]}{d^2 t} \right) \quad (1.19)$$

Con ese índice se obtiene la tensión del colapso:

$$U_{\text{colapso}} = U_{\text{corregida}}[n_{\text{pico}} + n_{\text{inflexion}}] \quad (1.20)$$

Se calculan los instantes correspondientes al 70 % y al 10 % de U_{colapso} sobre el flanco descendente, para extrapolar linealmente el instante de corte, para el nivel de U_{colapso} :

$$t_c = t_{70} + (U_{\text{colapso}} - U_{70}) \left(\frac{t_{10} - t_{70}}{U_{10} - U_{70}} \right) \quad (1.21)$$

Finalmente, se calcula el tiempo de corte respecto al origen virtual:

$$T_c = t_c - O_1 \quad (1.22)$$

Fase III: Convergencia, Filtrado y Parámetros Característicos

Independientemente de la ruta ejecutada en la Fase II, el algoritmo converge obteniendo una curva registrada y una curva base (ajustada numéricamente o escalada analíticamente), para avanzar a la secuencia final:

12. **Cálculo de curva residual:** Se obtiene como la diferencia algebraica entre la curva corregida ($U_{\text{corregida}}[n]$) y la curva base ($U_b[n]$) construida con los paráme-

tros obtenidos del ajuste:

$$R[n] = U_{\text{corregida}}[n] - U_b[n] \quad (1.23)$$

13. **Filtrado digital de fase cero:** Se obtiene la curva residual filtrada ($R_f[n]$), aplicando un filtro digital IIR a la curva residual ($R[n]$). Para reducir el ruido de muy alta frecuencia relacionado a la interferencia electromagnética, sin perder oscilaciones propias del ensayo.

14. **Curva de tensión de ensayo:** Se sintetiza la curva de ensayo ($U_t[n]$) sumando algebraicamente la curva base analítica con la residual filtrada:

$$U_t[n] = U_b[n] + R_f[n] \quad (1.24)$$

15. **Cálculo de parámetros característicos:** A partir de la curva de tensión de ensayo $U_t[n]$, se calculan los parámetros normativos. Dado que la señal procesada es de tiempo discreto, hay valores analíticos que pueden no corresponder con las muestras obtenidas, por lo que para encontrarlos se realiza una interpolación lineal.

- **Valor de cresta (U_t):** Es la amplitud máxima de la curva de tensión de ensayo $U_t[n]$:

$$U_t = \max(U_t[n]) \quad (1.25)$$

- **Tiempo de frente (T_1):** Se calcula como el cociente del intervalo temporal transcurrido entre los instantes t_{30} y t_{90} (correspondientes al 30 % y 90 % de U_t respectivamente), dividido por 0,6:

$$T_1 = \frac{t_{90} - t_{30}}{0,6} \quad (1.26)$$

- **Origen virtual (O_1):** El inicio ficticio de la descarga eléctrica se calcula:

$$O_1 = t_{30} - 0,5 \cdot (t_{90} - t_{30}) \quad (1.27)$$

- **Tiempo de cola (T_2):** El intervalo temporal entre el origen virtual O_1 y el instante t_{50} , correspondiente al 50 % de U_t en el flanco descendente de la onda, se calcula:

$$T_2 = t_{50} - O_1 \quad (1.28)$$

- **Sobreimpulso relativo (OS%):** Se calcula como la diferencia entre el valor extremo de la curva registrada (U_e):

$$U_e = \max(U_e[n]) \quad (1.29)$$

y el valor máximo de la curva base (U_b):

$$U_b = \max(U_b[n]) \quad (1.30)$$

expresado en porcentaje:

$$OS\% = \frac{U_e - U_b}{U_e} \cdot 100 \quad (1.31)$$

Grupo raíz (Archivo .h5)

Atributos globales: *Datos comunes a todos los registros*

Item: *Identificador del objeto a ensayar*

Cliente: *Nombre de la entidad solicitante*

Atenuaciones: *Factores de atenuación de CH1 y CH2*

Grupos de ondas *Impulso capturado (Onda_n / Referencia_n)*

Atributos particulares: *Metadatos propios de cada impulso*

Fecha: *Marca de tiempo de adquisición*

Temperatura: *Parámetro ambiental*

Humedad: *Parámetro ambiental*

Presión: *Parámetro ambiental*

Polaridad: *Sentido eléctrico del impulso (+/-)*

Valor_pico: *Tensión de cresta*

T1: *Tiempo de frente*

T2: *Tiempo de cola*

OS: *Sobreimpulso porcentual*

Grupo de Tiempos (/Time) [s]

raw: *Vector bruto a apartir de Δt*

alineado: *Vector alineado con el frente del impulso de referencia*

Grupo de CH1 (/CH1_Voltage) [V]

raw: *Vector de tensión bruta, magnitud real*

ensayo: *Vector de tensión de ensayo IEC 60060-1*

normalizado: *Vector normalizado ($V_{pico} = 1,0$)*

Grupo de CH2 (/CH2_Current) [A]

raw: *Vector de corriente bruta, magnitud real*

ensayo: *Copia del bruto (simetría de procesamiento con CH1)*

normalizado: *Vector normalizado ($I_{pico} = 1,0$)*

La etapa 1 de acondicionamiento analógico, es propia del montaje del ensayo, por lo que, no se afecta en ningún aspecto. La etapa 2 de digitalización, se modificó para

este proyecto respecto al montaje vigente, analizando entre las alternativas de osciloscopio disponibles en el laboratorio, cuál era el más indicado tanto por sus características eléctricas, como por su capacidad de comunicación con la PC. Mientras que, las etapas 3, 4 y 5 son desarrollo íntegro del presente trabajo.

Bibliografía

- [1] International Electrotechnical Commission, *Instruments and software used for measurement in high-voltage impulse tests - Part 2: Requirements for software for tests with impulse voltages and currents*, Geneva, Switzerland, 2013.
- [2] International Electrotechnical Commission, *High-voltage test techniques - Part 1: General definitions and test requirements*, Geneva, Switzerland, 2010.
- [3] Good Will Instrument Co., Ltd., *Digital Storage Oscilloscope GDS-1000A-U Series Programming Manual*, Part No. 82DS-112AUI01, New Taipei City, Taiwan, 2011. dirección: www.gwinstek.com.
- [4] Good Will Instrument Co., Ltd., *GDS-1000A-U Series USB Driver*, Accedido en 2026, GW Instek. dirección: <https://www.gwinstek.com>.
- [5] Good Will Instrument Co., Ltd., *Digital Storage Oscilloscope GDS-1000B Series Programming Manual*, New Taipei City, Taiwan, 2020. dirección: www.gwinstek.com.